

EEE3096S Website Maintainer Guide

Embedded Systems II — Course Website

Last updated: July 2026

This guide is intended for course staff responsible for maintaining the EEE3096S static course website. No web development experience is assumed. All tasks described here involve editing plain text files only.

1. Site Overview

The website is a static site built on the HTML5 UP Arcana template. “Static” means there is no server-side processing — the site is a collection of plain files (HTML, CSS, JavaScript, images, PDFs) that the browser downloads and displays directly.

The pages of the site are:

File	Purpose
<code>index.html</code>	Homepage — course description, course info table, announcements panel
<code>lectures.html</code>	Lecture slides and resources, organised by module and week
<code>practicals.html</code>	Practical assignments listing
<code>resources.html</code>	External links, board documentation, ARM assembly references
<code>archive.html</code>	Formula sheet plus past exams and class tests
<code>announcements.html</code>	Full announcements listing (all announcements)

1.1. Where files live

Content files are split across two top-level folders:

```

.
├── announcements/      Announcement text + manifest
│   ├── announcements.json
│   └── Announcement_00X.md
├── resources/         All course PDFs and downloads
│   ├── lecture content/    Lecture material (drives lectures.html)
│   │   └── ModuleN - Name/W#/  ...organised module, then week
│   ├── hardware documentation/ STM32 / ARM datasheets and manuals
│   └── past-papers/        Past exams, class tests, formula sheet
├── images/            Site imagery (banners, thumbnails)
└── assets/            Stylesheets, fonts, and JavaScript

```

2. Posting Announcements

Announcements are written as Markdown files and stored in `announcements/`. The homepage automatically displays the 3 most recent announcements. The full list appears on `announcements.html`.

2.1. Step 1: Write the announcement file

Create a new plain text file in `announcements/` named following this pattern:

```

Announcement_001.md
Announcement_002.md
Announcement_003.md

```

The number must increment by one each time. Use any plain text editor (Notepad, VSCode, etc.). Write the content using Markdown formatting:

```
## Practical 2 Due Date
```

```
Assignment P2 is due **Friday 14 March at 5pm**. Submit via Vula.
```

```
Late submissions will be penalised 10% per day.
```

Key Markdown formatting rules:

- `## Heading` — section heading
- `**bold text**` — bold
- `*italic text*` — italic
- A blank line between paragraphs creates a new paragraph

2.2. Step 2: Register the file in the manifest

Open `announcements/announcements.json` and add the new filename to the bottom of the list:

```
[  
  "Announcement_001.md",  
  "Announcement_002.md",  
  "Announcement_003.md"  
]
```

Important: the manifest file controls which announcements are displayed. If you add a `.md` file to `announcements/` but forget to add it to `announcements.json`, it will not appear on the site.

The homepage always shows the last 3 entries in the list (i.e. the bottom 3), so newer announcements should always be appended to the bottom.

2.3. Step 3: Deploy

Save both files and upload/push them to the server as you normally would for any site update. No other changes are required.

3. When the Course Is Inactive

When the course is not running (e.g. between semesters), you may want to suppress certain content without deleting it. The recommended approach is to comment it out in the HTML.

HTML comments use this syntax:

```
<!-- This content is hidden -->
```

Anything between `<!--` and `-->` is ignored by the browser.

3.1. Suppressing the announcements panel

If there are no active announcements, comment out the announcements `<script>` in `index.html`, and uncomment the `<script>` below it to display a message about the course being inactive:

```

<!--#####
    Comment out from here if course is inactive -->
    <!-- Script fetches and parses most recent 3 announcements from
    announcements.json manifest-->

    <script>
    fetch('announcements/announcements.json')
      .then(response => response.json())
      .then(files => {
        const recent = files.slice(-3).reverse();
        return Promise.all(
          recent.map(filename =>
            fetch('announcements/' + filename).then(r => r.text())
          )
        );
      })
      .then(texts => {
        const html = texts.map(t =>
          '<div style="background: #f9f9f9; border-left: 4px solid #ccc;
padding: 1em 1.5em; margin-bottom: 1.5em; max-width: 700px; word-wrap:
break-word; overflow-wrap: break-word;">'

          + marked.parse(t)
          + '</div>'
        ).join('');
        document.getElementById('announcements-content').innerHTML = html;
      })
      .catch(() => {
        document.getElementById('announcements-content').innerHTML = '<p>No
announcements at this time.</p>';
      });
    </script>

<!--#####
    Comment out until here if inactive -->

    <!--Uncomment if course is inactive:
    <script>
      document.getElementById('announcements-content').innerHTML =
"Course currently inactive"
    </script>
    -->

```

4. Updating Course Content

4.1. Updating Course Staff

In `index.html`, the course staff details are displayed. To update these details, modify the following section in the `index.html` file:

```
<!-- #####  
        Modify this section to update course staff-->  
  
        <h3 style="font-size: 18px; color:green"> Lecturers: </h3>  
        Prof. Simon Winberg (Head Lecturer)<br>  
        Email:<a href="mailto:simon.winberg@uct.ac.za">  
simon.winberg@uct.ac.za </a> <br>  
        Office: Menzies 6.13 <br> <br>  
  
        Travimadox Webb (Co-Lecturer )<br>  
        Email:<a href="mailto:WBBTRA001@myuct.ac.za">  
WBBTRA001@myuct.ac.za </a>  
        <br> <br>  
  
        <h3 style="font-size: 18px; color:green"> Teaching Assistant:  
</h3>  
        Maisha Magavha  
        Email:<a href="mailto:MGVMAI001@myuct.ac.za">  
MGVMAI001@myuct.ac.za </a> <br> <br>  
  
        <h3 style="font-size: 18px; color:green"> Tutors: </h3>  
        <!-- TODO-->  
  
<!-- ##### -->
```

4.2. Adding Lecture Material

Lecture material is organised in the folder `resources/lecture content/`, first by module and then by week:

```
resources/lecture content/  
├─ Module1 - Embedded Systems/  
│   ├── W1/  
│   │   ├── EEE3096S-L01-Intro to ES - Copy.pdf  
│   │   └─ P1_code.zip  
│   └─ W2/  
└─ Module2 - Assembly & C/  
    └─ W3/
```

The `lectures.html` page displays this material. Within each week, lecture **slide decks** are shown as clickable cover-page thumbnails, followed by a list of **additional resources** (exercises, code, datasheets). Each week and module also has a **Download** button that bundles its files into a single zip. Two things happen automatically in the visitor's browser, so you do not have to prepare them yourself:

- **Thumbnails** are drawn from the first page of each slide PDF by pdf.js (`assets/js/pdf.min.js`). You do not need to create or upload thumbnail images.
- **Download buttons** are built by JSZip (`assets/js/jszip.min.js`), driven by `assets/js/lectures.js` . The script simply collects every link inside the surrounding week or module block, so there is no file list to maintain — anything you add is picked up automatically.

4.2.1. Step 1: Add the file

Place the file in the correct `Module/Week` folder. If you are starting a new week or module, create the folder using the same naming pattern as the existing ones.

4.2.2. Step 2: Add an entry to `lectures.html`

Adding the file to the folder is not enough on its own — it must also be listed in `lectures.html` . Which snippet you use depends on the type of file.

For a lecture slide deck, paste a slide card inside the `<div class="row slides">` for that week:

```
<div class="col-2 col-3-narrow col-4-narrower col-6-mobile col-12-mobilep">
  <a class="slide" href="resources/lecture%20content/Module1%20-%20Embedded%20Systems/W1/EEE3096S-L01-Intro%20to%20ES.pdf" download>
    <canvas data-pdf="resources/lecture%20content/Module1%20-%20Embedded%20Systems/W1/EEE3096S-L01-Intro%20to%20ES.pdf" aria-hidden="true"></canvas>
    <span class="slide-title">L01 &mdash; Intro to Embedded Systems</span>
  </a>
</div>
```

The path appears twice and must match in both places: the `href` is what downloads when the card is clicked, and the `data-pdf` is what the thumbnail is rendered from. The `<span class="slide-title">` is the caption shown under the thumbnail.

For any other resource (take-home activities, code archives, reference PDFs), add a list item inside that week's `<ul class="resources">` :

```
<li><a href="resources/lecture%20content/Module1%20-%20Embedded%20Systems/W1/P1_code.zip">P1 &mdash; Code (ZIP)</a></li>
```

Paths must be percent-encoded. The folder names contain spaces and other characters that are not valid in a URL, so they must be substituted:

- a space becomes `%20`
- `&` becomes `%26`
- `+` becomes `%2B`

So the folder `Module2 - Assembly & C` is written `Module2%20-%20Assembly%20%26%20C`. A mis-encoded path produces a broken link and a blank thumbnail.

Copying an existing entry inside `lectures.html` and editing only the filename is the safest way to get this right — and is safer than copying out of this PDF, where long paths are broken across lines for printing.

4.2.3. Adding a new week or module

To add a week, copy an entire `<div class="lecture-week">` block, then update its `<h4>` heading and the `data-zipname` on its download button. To add a module, copy an entire `<section class="lecture-module">` block and update its `<h3>` heading and `data-zipname`.

The `data-zipname` value is the filename of the zip produced by that block's download button, so keep it descriptive (e.g. `Module 1 - Week 3`).

4.3. Adding a practical

`practicals.html` is currently a placeholder page with no practical entries on it. When adding the first practical, follow the same approach as the lectures page: place the brief in `resources/`, then add a linked entry inside the page's `<section>` block.

4.4. Updating the course handout link

In `index.html`, find the link to the course handout in the welcome text and update the `href`:

```
<a href="/resources/EEE3096S_2025 Course Handout.pdf">Course Handout </a>
```

Place the new PDF in `resources/` and update the filename in the link. Note that the STM32CubeIDE setup guide is linked further down the same page, and uses a percent-encoded path:

```
<a href="resources/CUBE%20IDE%20setup.pdf">STM32CubeIDE Setup Guide</a>
```

5. Important Notes

- **Never edit `assets/css/main.css`** unless you are confident in CSS. It is generated from the SASS source in `assets/sass/`, so hand edits are lost when it is regenerated. Small visual changes are better made using inline `style=""` attributes on individual elements.
- **The nav bar is not shared** — it is manually duplicated in every HTML file. If you add or remove a nav item, you must update all six pages.
- **Announcements are ordered by position in `announcements.json`**, not by file date. Always append new entries to the bottom of the list.
- **Lecture material must be listed in `lectures.html` by hand.** Dropping a file into a `Module/Week` folder does not make it appear on the site — the browser cannot see inside folders on the server. Only the download buttons and thumbnails are automatic, and only for entries already listed on the page.
- **Percent-encode paths under `resources/lecture content/`** — spaces become `%20`, `&` becomes `%26`, `+` becomes `%2B`.
- **The site requires an HTTP server to function correctly.** Opening HTML files directly from the file system (double-clicking) will cause announcements to fail to load, and will also leave lecture thumbnails blank and break the download buttons. Use Live Server in VSCode during local editing, or deploy to a proper web host. The published site is served over HTTP, so these features work there.